CS3400 – Data Structures and Algorithms
Mini Project Technical Report

Group Type A – Campus Facilities

Institution	National University of Lesotho (NUL)
Course	CS3400 Data Structures and Algorithms
Language	C++17
Group Type	Type A – Campus Facilities
System	Campus Resource & Activity Management

This report documents the design, implementation and analysis of the complete system developed by the group, covering all required modules and data structures.

1. Introduction

1.1 Problem Description

The National University of Lesotho manages a large number of physical campus resources including lecture rooms, laboratories, computer labs, offices, libraries, meeting rooms, sports facilities and equipment. These resources are distributed across multiple buildings (Science Building, BTM, FTF, Academic Building, Administration, etc.). Manual management of availability, booking, prioritization of requests, and navigation between facilities is inefficient and error-prone. The system addresses this by providing an integrated software solution that stores, searches, prioritizes and navigates campus resources using fundamental data structures and algorithms.

1.2 Project Objectives

The main objectives of the project are:

- Implement custom data structures (Linked List, Queue, Stack, Priority Queue / Heap, BST, HashTable, Graph) in C++17 with proper encapsulation, dynamic memory and pointers.
- Provide complete CRUD operations for campus resources (lecture rooms, labs, offices, etc.).
- Support prioritized request processing and undo capability.
- Enable efficient search and sorting of resources.
- Model hierarchical organization of facilities and campus navigation with shortest-path support.
- Persist data to files and demonstrate the full system through a menu-driven interface.
- Ensure every group member understands the complete system while maintaining clear role allocation.

1.3 Scope of the System

The system focuses on Campus Facilities (Group Type A). It manages lecture rooms, laboratories, computer labs, offices, libraries, meeting rooms, sports facilities and equipment located in

buildings such as the Library, Computer Science Building, Administration, Student Affairs, Laboratories and Academic Building. The system includes resource management, request handling, searching, sorting, tree and hash-based retrieval, graph-based campus navigation, timetable management for academic rooms, and file persistence. It does not include full university timetable generation, financial billing, or multi-user concurrent access.

2. System Design

2.1 Overall System Architecture

The system follows a layered design centered on a CampusManager controller. The controller coordinates three logical layers of responsibility corresponding to the three students, while presenting a single unified menu to the user.

Architecture (high level):

User Interface (main menu) → CampusManager → [ResourceManager + LinkedList + Sorting + FileHandler] (Student A) | [RequestProcessor + Queue + Stack + PriorityQueue] (Student B) | [BST +
CategoryTree + HashTable + Graph] (Student C) | [BookingManager + Timetable]

When a resource is added, CampusManager inserts it into the LinkedList (primary store), the BST (ordered by ID) and the Hash Table (fast lookup) so that all three structures remain consistent. The Graph is pre-loaded with realistic campus locations and pathways but remains editable. Timetable entries are stored separately and persisted to a text file under admin control. **2.2 Class Design (Major Classes)**

Class	Responsibility	Owner
Resource	Represents a campus facility (ID, name, type, location, capacity, status)	A
LinkedList<T>	Primary dynamic storage for resources and bookings	A
ResourceManager	CRUD operations over the resource LinkedList	A
FileHandler	Load/save resources from/to text files	A
Booking / BookingManager	Timetable entries and conflict checking	A
Request	User request with priority and status	B
Queue<T>	FIFO processing of normal requests	B
Stack<T>	Processed-request history / undo support	B
PriorityQueue<T>	Max-heap for emergency/high priority requests	B
RequestProcessor	Coordinates normal and priority request flows	B
BST	Binary Search Tree ordered by Resource ID	C
CategoryTree	Hierarchical organisation of facility categories	C
HashTable	Separate-chaining hash table for O(1) average lookup	C
Graph	Campus map: locations + pathways, BFS/DFS/Dijkstra	C
CampusManager	Central controller; keeps structures synchronized	All

2.3 Major Data Structures Summary

The system deliberately implements its own Linked List, Queue, Stack, binary max-heap Priority Queue,
Binary Search Tree, Hash Table (separate chaining) and Graph (adjacency list). Only limited STL helpers (std::string, std::vector, std::queue for level-order) are used. This satisfies the course requirement to demonstrate encapsulation, constructors/destructors, pointers and dynamic memory management.

3. Data Structure Selection

Linked List

Problem solved: Primary storage of an arbitrary and changing number of campus resources.

Why selected: Supports efficient insertion and deletion at the ends and sequential traversal without a fixed capacity. Matches the dynamic nature of campus inventory.

Alternatives considered: Array (fixed size or expensive resizing); std::vector (disallowed as primary store).

Queue

Problem solved: FIFO processing of ordinary (non-urgent) resource requests.

Why selected: Guarantees fair, first-come-first-served order for normal requests. Implemented as a circular linked list.

Alternatives considered: Stack (wrong order); Priority Queue (unnecessary overhead for normal requests).

Stack

Problem solved: History of processed requests and potential undo support.

Why selected: LIFO order naturally models “last action can be reversed first”.

Alternatives considered: Queue (wrong order for undo).

Priority Queue (Binary Max-Heap)

Problem solved: Processing of urgent requests (HIGH / EMERGENCY) before normal ones.

Why selected: Extract-max is O(log n); insertion is O(log n). Guarantees the highest-priority request is always handled next.

Alternatives considered: Sorted Linked List (O(n) insert); unsorted list (O(n) extract-max).

Binary Search Tree (BST)

Problem solved: Keeping resources ordered by ID and supporting ordered traversals.

Why selected: In-order traversal yields sorted order; average search/insert/delete is O(log n). Complements linear search and hashing.

Alternatives considered: Sorted array (expensive inserts); Hash Table alone (no ordered enumeration).

Hash Table (Separate Chaining)

Problem solved: Very fast average-case lookup of a resource by ID.

Why selected: Average O(1) search. Polynomial rolling hash + prime table size (101) reduces collisions. Demonstrates hashing vs tree vs list trade-offs.

Alternatives considered: Open addressing (harder deletion); BST alone (no O(1) average).

Graph (Adjacency List)

Problem solved: Modelling campus locations and pathways; supporting navigation and shortest path.

Why selected: Natural representation of a campus map. Enables BFS, DFS and Dijkstra. Pre-loaded with realistic NUL locations yet remains editable.

Alternatives considered: Adjacency matrix (wastes space for sparse campus graph).

Category Tree (n-ary)

Problem solved: Hierarchical organization of facility types (Campus → Academic → Laboratories, etc.).

Why selected: Mirrors real-world classification and supports clear visualization of the facility hierarchy.

Alternatives considered: Flat list (loses hierarchy); BST (ordered by ID, not by category).

4. Algorithms

4.1 Searching

Linear Search – Scans the LinkedList (or an array produced from it) from the beginning until the target Resource ID is found or the end is reached. Used as a baseline and inside each Hash Table chain.

Binary Search – Performed on a sorted array of resources (after Merge Sort). Requires the collection to be ordered by ID. Offers O(log n) search when the array is already sorted.

BST Search – Recursive comparison of the target ID with the current node; descends left or right. Average O(log n), worst O(n) if the tree becomes skewed.

Hash Table Search – Computes the bucket index via a polynomial rolling hash, then performs a short linear search inside the chain. Average O(1).

4.2 Sorting

Merge Sort – Primary sorting algorithm. Stable, guaranteed O(n log n) time. Used to sort resources by ID, name or capacity. The implementation uses a temporary array for the merge step.

Insertion Sort – Also implemented for comparison and for nearly-sorted or small collections. Best case O(n), worst case $O(n^2)$.

4.3 Tree Operations

BST supports recursive insert, search and delete (with the standard three cases: leaf, one child, two children using in-order successor). Four traversals are provided: in-order (sorted output), pre-order, post-order and level-order (using a helper queue). The Category Tree supports hierarchical display and addition of new categories under an existing parent.

4.4 Graph Traversal and Shortest Path

BFS – Explores locations level by level from a start vertex using a queue. Useful for finding the fewest-hop path and for reachability.

DFS – Recursive depth-first exploration. Used for connectivity and for listing connected components.

Dijkstra's Algorithm – Computes the shortest path (by distance in metres) between two campus locations. Implemented with a simple $O(V^2)$ selection of the next closest vertex (acceptable for the modest size of a campus graph). Reconstructs the path via a predecessor array.

5. Complexity Analysis

The table below summarizes the time complexity of the most important operations used in the system. n = number of resources (or vertices for graph algorithms).

Operation	Best	Average	Worst	Notes
Add resource (LinkedList end)	O(1)	O(1)	O(1)	Tail pointer
Search by ID (LinkedList)	O(1)	O(n)	O(n)	Linear scan
Search by ID (Hash Table)	O(1)	O(1)	O(n)	Chain length
Search by ID (BST)	O(1)	O(log n)	O(n)	Skewed tree
Enqueue / Dequeue	O(1)	O(1)	O(1)	Circular list
Push / Pop (Stack)	O(1)	O(1)	O(1)	Linked stack
Heap insert	O(1)	O(log n)	O(log n)	Bubble up
Heap extract-max	O(log n)	O(log n)	O(log n)	Heapify
BST insert	O(log n)	O(log n)	O(n)	Balanced vs skewed
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Guaranteed
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Nearly sorted best
BFS / DFS	O(V+E)	O(V+E)	O(V+E)	Adjacency list
Dijkstra (simple)	$O(V^2)$	$O(V^2)$	$O(V^2)$	Dense selection

The design deliberately offers three different search methods (linear, tree, hash) so that their practical performance differences can be observed and discussed during the demonstration.

6. Testing

6.1 Test Categories

Testing covered normal cases, boundary cases, invalid input and large data.

Category	Example Test	Expected Result
Normal	Add resource ICT-LAB, search via Hash and BST	Found in all three structures
Normal	Submit priority request then process it	Highest priority extracted first
Normal	Dijkstra from Library to Labs	Path and total distance displayed
Boundary	Search / traverse empty BST or empty Graph	Graceful “empty” message
Boundary	Process request when queues are empty	“No requests” message
Invalid	Add resource with duplicate ID	Rejected with message
Invalid	Search non-existent ID	Not found in all structures
Invalid	Wrong admin password for timetable	Access denied
Large data	dataGenerator loads 200+ rooms at start	All rooms searchable and sortable
Large data	In-order traversal of BST with 200+ nodes	Sorted list produced without crash

6.2 Sample Data

On startup the system generates a realistic set of NUL facilities via dataGenerator (Science Building labs and offices, BTM lecture rooms, FTF rooms, ICT Lab, Cisco Lab, Administration, Netherlands Hall, Cafeteria, etc.). The Graph is pre-loaded with locations such as Library, CS Building, Academic Building, Labs, Administration, Student Affairs, Sports Complex, BTM, FTF and Netherlands Hall, together with pathway distances. Timetable entries can be added by an administrator and are stored in data/timetable.txt.

7. Challenges and Lessons Learned

7.1 Technical Challenges

- Integrating three independently developed components required careful alignment of the Resourceinterface (getID vs getId, const-correctness, Status enum conflicts between Resource and Request).

- Keeping LinkedList, BST and Hash Table synchronised on every insert and delete demanded a singlecontroller (CampusManager) that owns the update logic.
- Template LinkedList explicit instantiations and header-only Queue/Stack/PriorityQueue neededconsistent include paths and compilation order.
- Windows console encoding of tree-drawing characters and path spaces required practicalwork-arounds.

7.2 Design Challenges

- Deciding which structure is the “primary” store (LinkedList) and which are secondary indexes (BST,Hash) while still meeting the course requirement to implement and demonstrate every structure.
- Balancing a rich feature set (timetable, pre-loaded data, editable graph) against the need for a cleardemonstration path that covers all ten required modules.

7.3 Team Collaboration

Work was divided by data-structure ownership (A: core storage and sorting; B: requests and priority; C: tree, hashing and graph). Integration was performed late, which surfaced interface mismatches. A shared CampusManager and a single main menu proved essential. Regular communication about method names and ownership of the Resource class would have reduced rework.

7.4 Important Lessons

- Agreeing on a common Resource interface and naming conventions at the start of the project iscritical.
- Implementing each data structure in isolation first, then integrating, makes debugging easier.
- Providing multiple search methods (list, tree, hash) is an excellent way to illustrate the practicalimpact of asymptotic complexity.
- A thin controller layer that keeps secondary indexes consistent is a valuable architectural patterneven in a student project.
- Pre-loading realistic data and a campus map makes the demonstration far more convincing thanempty structures.